

Name: Aman Singh

Decision Tables — Explained

1. Login Validation

1. Login Validation				
Conditions	Rules			
	R1	R2	R3	R4
Username entered?	Y	Y	N	N
Password entered?	Y	N	Y	N
Valid credentials?	Y	-	-	-
Actions				
Login Successful	X	-	-	-
Show "Enter Password" error	-	X	-	-
Show "Enter Username" error	-	-	X	-
Show "Invalid Credentials" error	-	-	-	X

What this table checks

This decision table decides what happens when a user tries to log in, based on two inputs: whether the username was entered, whether the password was entered, and (only when both are present) whether the credentials are valid.

Rule-by-rule explanation

- **R1** — Username entered (Y), Password entered (Y), Credentials valid (Y) → **Login Successful**. This is the normal happy-path case.
- **R2** — Username entered (Y), Password entered (N) → **"Enter Password" error** is shown. The credential-validity condition doesn't matter here (marked "-") because the system can't check credentials without a password.
- **R3** — Username entered (N), Password entered (Y) → **"Enter Username" error** is shown, again regardless of credential validity, since username is missing.
- **R4** — Username entered (N), Password entered (N) → **"Invalid Credentials" error** is shown as the general fallback when both fields are empty.

Key insight: the "Valid credentials?" condition is only evaluated in R1, because it's only meaningful once both the username and password have actually been entered — this is a classic use of the "don't care" (-) marker in decision tables to avoid redundant rules.

2. Online Shopping – Apply Coupon

2. Online Shopping – Apply Coupon				
Conditions	Rules			
	R1	R2	R3	R4
Cart value > Min. amount?	Y	Y	N	-
Coupon valid?	Y	N	-	-
Coupon not expired?	Y	Y	-	-
Actions				
Apply discount	X	-	-	-
Show "Invalid Coupon"	-	X	-	-
Show "Minimum amount not met"	-	-	X	-
Do not apply coupon	-	-	-	X

What this table checks

This table governs whether a discount coupon is applied at checkout, based on the cart value versus a minimum threshold, whether the coupon code itself is valid, and whether it has expired.

Rule-by-rule explanation

- **R1** — Cart value above minimum (Y), Coupon valid (Y), Not expired (Y) → **Apply discount**. All conditions favorable, so the discount is applied.
- **R2** — Cart value above minimum (Y), Coupon valid (N) → **"Invalid Coupon"** message is shown. The expiry condition is irrelevant once the coupon itself is invalid, so it's marked "-".
- **R3** — Cart value *not* above minimum (N) → **"Minimum amount not met"** message is shown, regardless of coupon validity or expiry — cart value is checked first and short-circuits the rest.
- **R4** — Catch-all case (marked "-" for cart value) → **Do not apply coupon**. This acts as a default/else rule covering any remaining combination not captured above.

Key insight: the conditions are effectively evaluated in priority order — cart value first, then coupon validity, then expiry — which is a common pattern to keep the table compact instead of listing all $2^3 = 8$ possible combinations.

3. ATM Cash Withdrawal

3. ATM Cash Withdrawal

Conditions	Rules			
	R1	R2	R3	R4
Card inserted?	Y	Y	Y	N
Correct PIN?	Y	N	-	-
Sufficient balance?	Y	Y	N	-
Actions				
Dispense cash	X	-	-	-
Show "Incorrect PIN"	-	X	-	-
Show "Insufficient Balance"	-	-	X	-
Eject card	-	-	-	X

What this table checks

This table models the standard ATM withdrawal flow, based on whether a card was inserted, whether the correct PIN was entered, and whether the account has sufficient balance.

Rule-by-rule explanation

- **R1** — Card inserted (Y), Correct PIN (Y), Sufficient balance (Y) → **Dispense cash**. The complete happy path.
- **R2** — Card inserted (Y), Correct PIN (N) → **"Incorrect PIN"** error is shown; the balance check never happens because the transaction is already rejected at the PIN stage.
- **R3** — Card inserted (Y), Correct PIN implied ("- i.e. PIN accepted), Sufficient balance (N) → **"Insufficient Balance"** error is shown.
- **R4** — Card not inserted (N) → the machine simply **ejects the card** (or rather, prompts/ends the session), since no transaction can proceed at all without a card.

Key insight: this table shows a strict sequential dependency — PIN is only checked after the card is inserted, and balance is only checked after the PIN is correct — so each failure condition short-circuits the remaining checks.

4. Leave Approval (Employee)

4. Leave Approval (Employee)

Conditions	Rules			
	R1	R2	R3	R4
Leave days ≤ 2?	Y	N	-	-
Leave days > 2?	-	Y	Y	-
Manager available?	Y	-	Y	N
HR available?	-	-	-	Y
Actions				
Manager approves	X	-	-	-
HR approves	-	X	-	-
Both approve	-	-	X	-
Leave cannot be approved	-	-	-	X

What this table checks

This table decides who approves an employee's leave request, based on the number of leave days requested and the availability of the manager and/or HR.

Rule-by-rule explanation

- **R1** — Leave days ≤ 2 (Y), Manager available (Y) → **Manager approves**. Short leaves only need the manager's sign-off.
- **R2** — Leave days > 2 (Y), Manager availability not relevant ("-") → **HR approves**. This suggests that for longer leaves, HR alone can approve in this branch of the logic.
- **R3** — Leave days > 2 (Y), Manager available (Y) → **Both approve**. Longer leaves need dual sign-off from both manager and HR when the manager is available.
- **R4** — Manager not available (N), HR available (Y) → **Leave cannot be approved**. Even though HR is available, without the manager the request is blocked in this rule — highlighting that manager involvement is treated as mandatory for this path.

Key insight: R2 and R3 both apply when leave days > 2, but differ by manager availability — this shows how a single condition value ("Y") can lead to different outcomes depending on a second condition, which is exactly what decision tables are good at capturing clearly.

5. Restaurant Order Discount

5. Restaurant Order Discount

Conditions	Rules			
	R1	R2	R3	R4
Order amount > 1000?	Y	Y	N	-
Customer is member?	Y	N	-	-
Special day (Weekday)?	Y	N	Y	N
Actions				
Apply 20% discount	X	-	-	-
Apply 10% discount	-	X	-	-
Apply 5% discount	-	-	X	-
No discount	-	-	-	X

What this table checks

This table calculates the discount percentage applied to a restaurant order, based on the order amount, whether the customer is a member, and whether the order falls on a special (weekday) day.

Rule-by-rule explanation

- **R1** — Order > \$1000 (Y), Member (Y), Special day (Y) → **Apply 20% discount**. The maximum discount, requiring all three favorable conditions.
- **R2** — Order > \$1000 (Y), Member (N), Special day (N) → **Apply 10% discount**. A moderate discount purely for a high-value order, even without membership or a special day.
- **R3** — Order ≤ \$1000 (N), Special day (Y) → **Apply 5% discount**. A small discount is still given on special days even for lower-value orders.
- **R4** — Order ≤ \$1000 (N), Special day (N) → **No discount**. The baseline case where none of the favorable conditions are met.

Key insight: the discount tiers are structured so that order value, membership, and timing each independently contribute to the final discount — but the table only lists the specific combinations relevant to real-world pricing rules rather than all possible combinations.